



Advancing Cloud Scalability

Application autoscaling in K8s environment

Abhilash Gaddala

Ibtihajulla Sha Shaik

Suneel Seelam

Project Overview: Exploring Adaptive Autoscaling

Adaptive Step-Size Algorithm

Implementation of advanced autoscaling logic for Kubernetes pods.

Lambda Function Integration

Algorithm deployed as a serverless Lambda for dynamic execution to scale Kubernetes worker nodes

CloudWatch for Performance

Comprehensive data collection and analysis for evaluation.



Kubernetes Workflow: Scaling a Web Application

01

Cluster Setup

Initialization of Kubernetes environment on EC2.

02

K8s Apps Deployment

Containerized web app deployed as Pods.

03

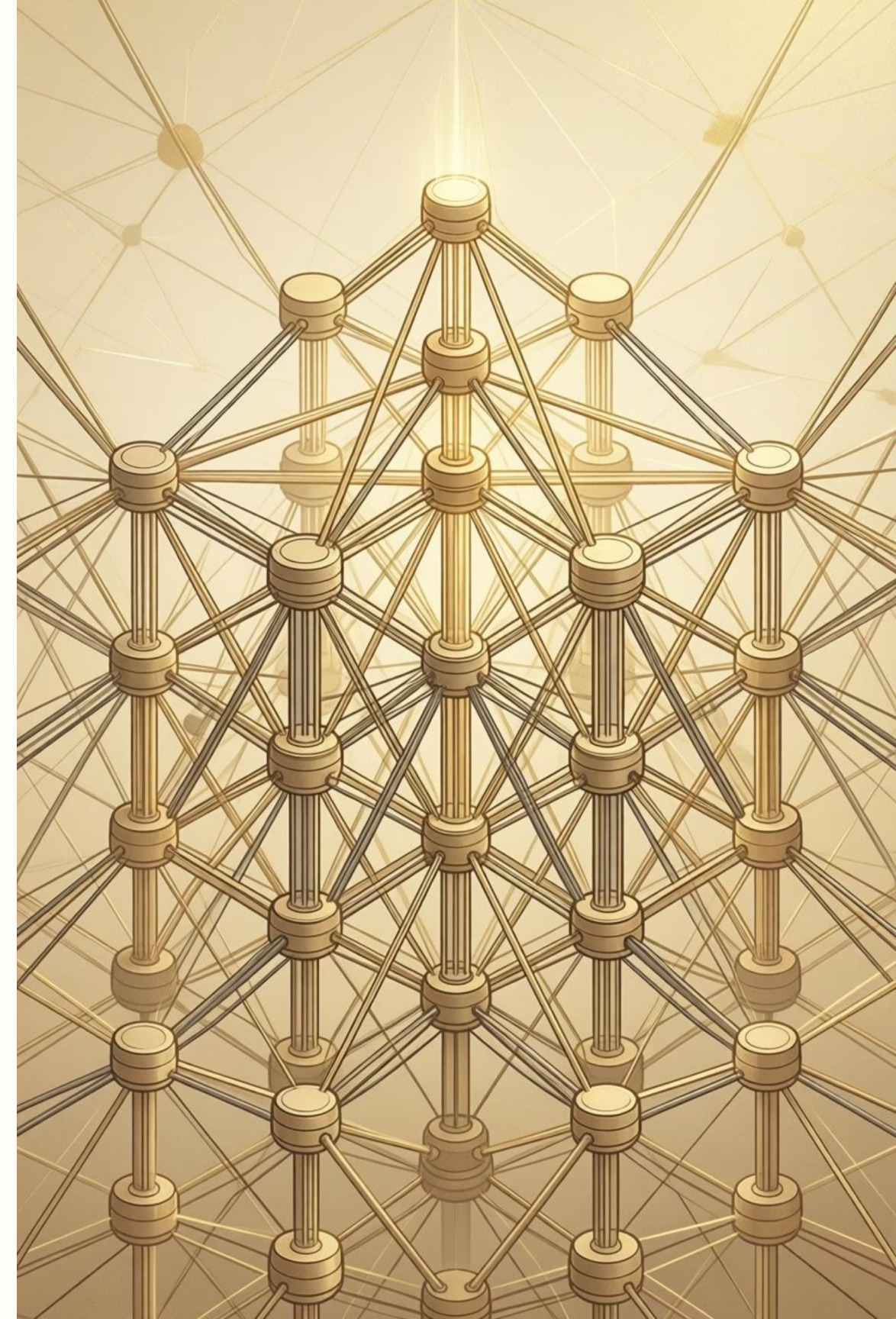
Autoscaling Module

Configuration and testing of scaling logic.

04

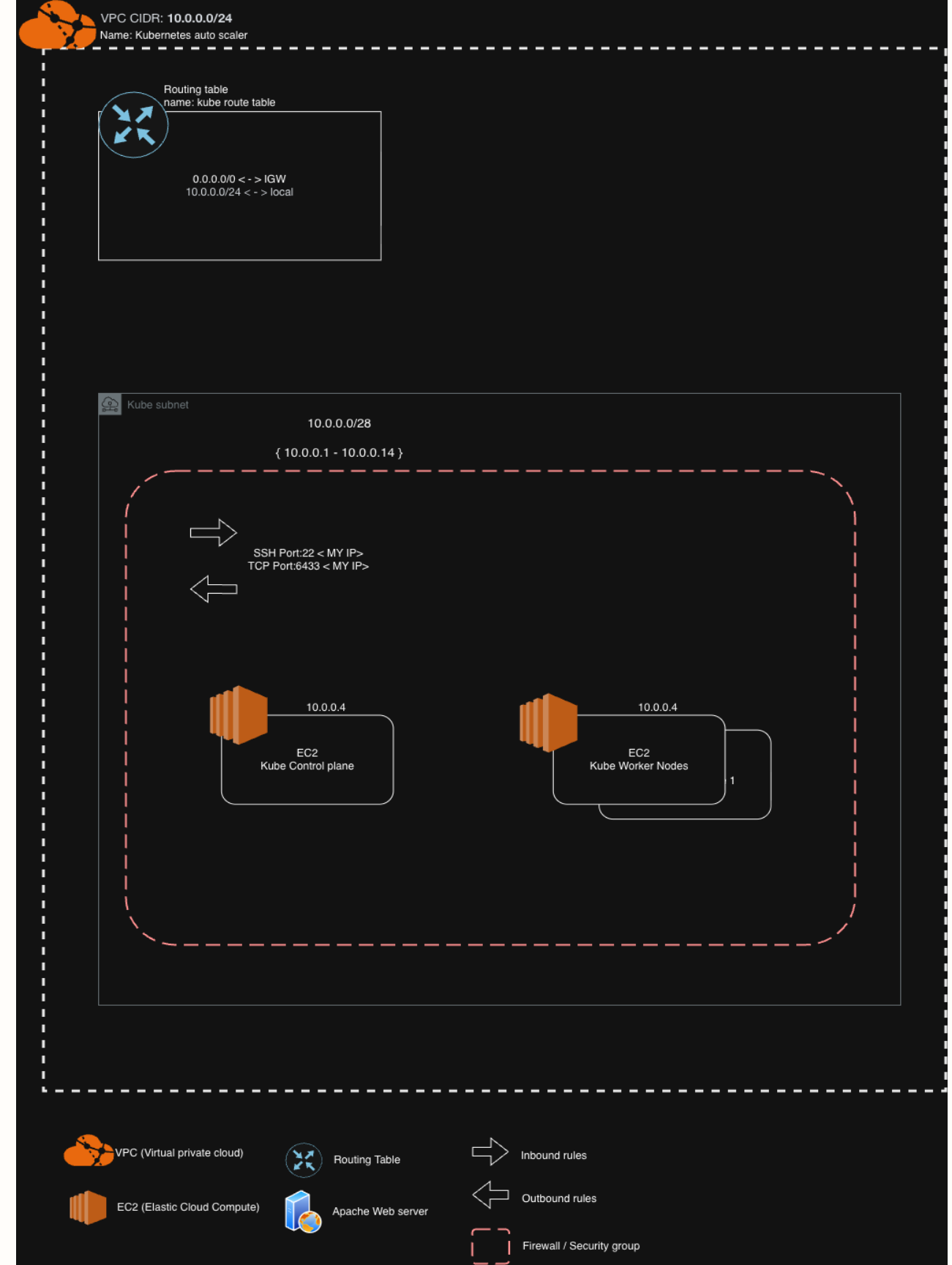
Load Simulation

Verification of dynamic scaling behavior.



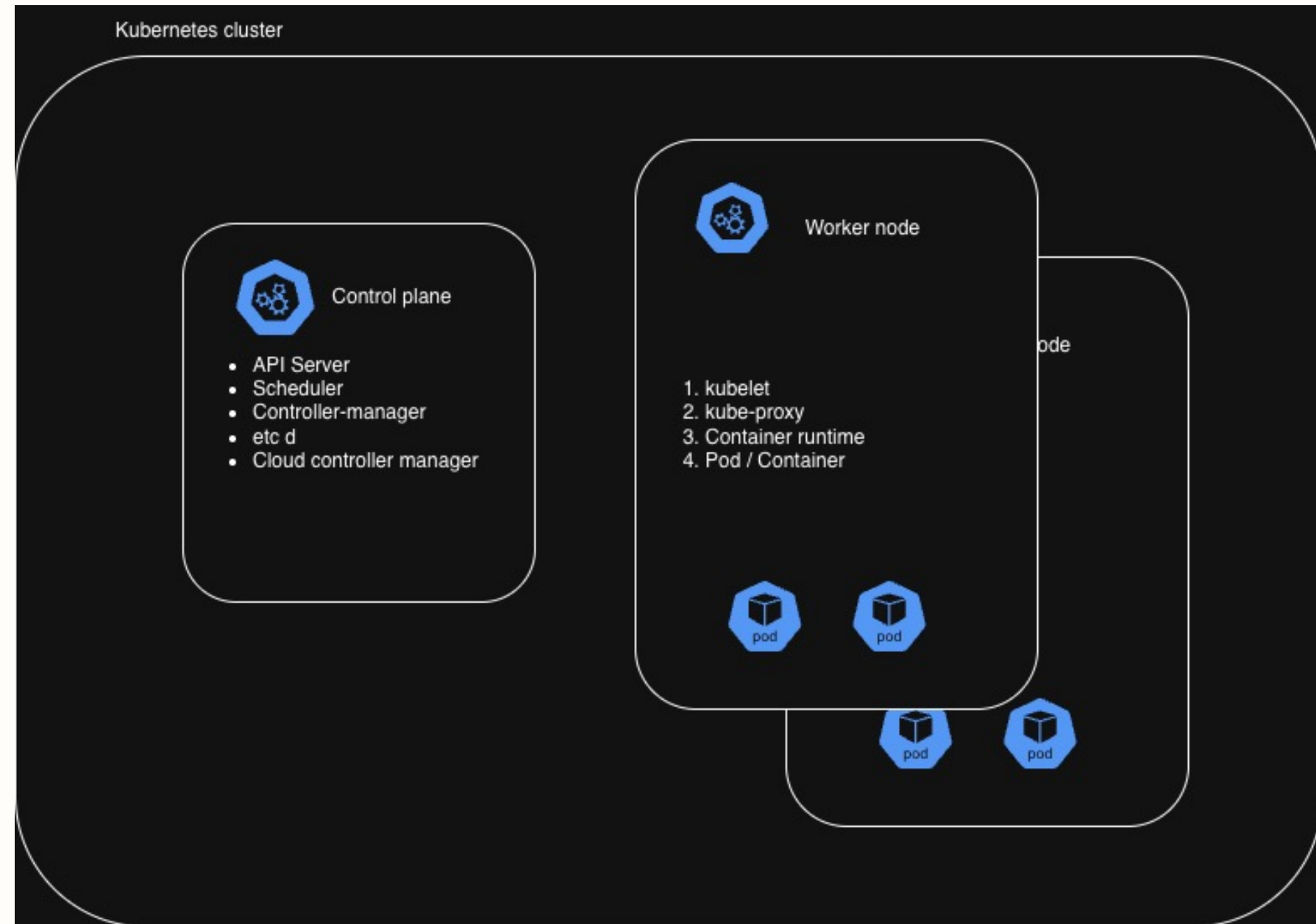
Cluster setup

- VPC
- Routing table
- Subnet
- Security group
- EC2
- Cloud watch (Monitoring)
- AMI (Kubernetes worker node)
- Auto Scaler
- RDS
- Cloud watch



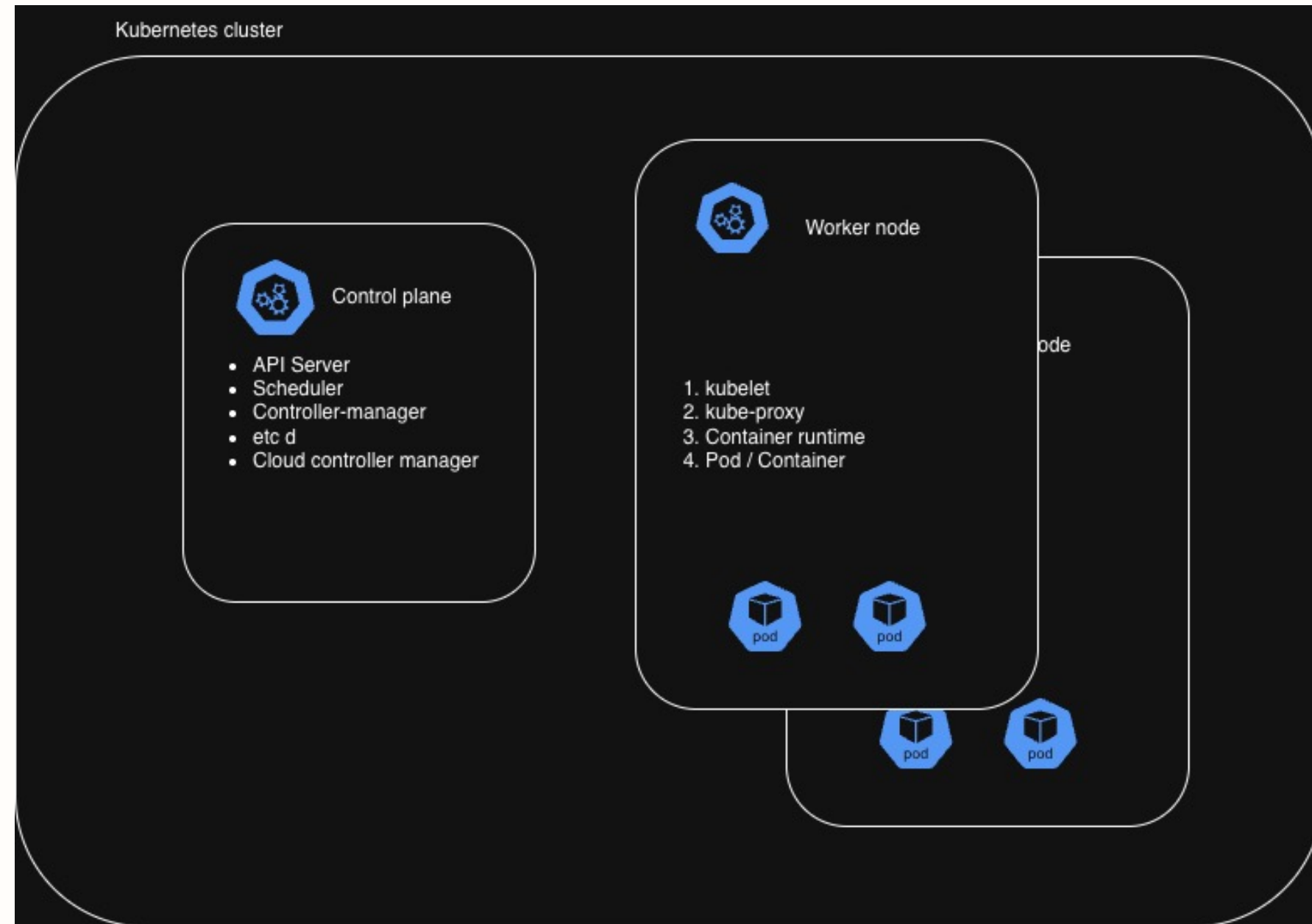
K8s cluster configuration

- Control plane
- Worker nodes (AMI)
- Auto scaler + Cloud watch



K8s apps deployment

- Helm manifest
- Dockerized application
- K8s Pod definition with replica set
- K8s Service
- K8s HPA (Horizontal pod autoscaler)



Application deployment

Deployment definition

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: webserver-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: webserver
  template:
    metadata:
      labels:
        app: webserver
    spec:
      containers:
        - name: webserver-deployment
          image: suneelseelam33/userregister-app:v1
          ports:
            - containerPort: 80
```

Service definition

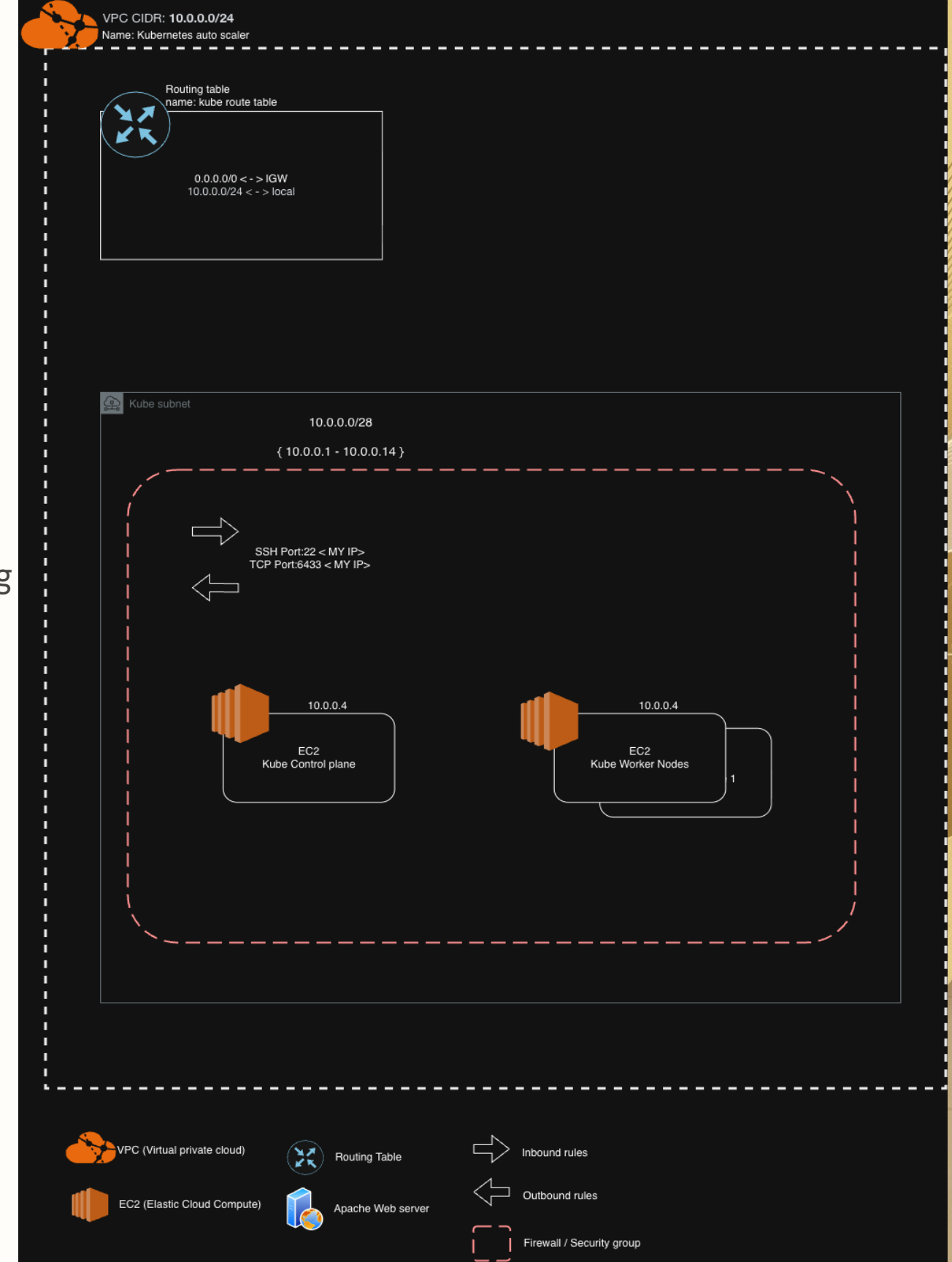
```
apiVersion: v1
kind: Service
metadata:
  name: webserver-service
spec:
  selector:
    app: webserver
  type: NodePort
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
      nodePort: 30080
```

HPA definition

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: webserver-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: webserver-deployment
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 50
```

Setup Installation:

This project focuses on implementing an adaptive step-size algorithm, a sophisticated method to dynamically adjust Kubernetes pods autoscaling and AWS EC2 instances to increase Kubernetes worker nodes. Unlike traditional methods, it learns and adapts to workload patterns.



Application runtime in K8s

Default service configuration running with 2 pods

```
ubuntu@k8s-master:~/deployment$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
webserver-deployment-5bd4f98fdb-27fg4 1/1     Running   0           73m
webserver-deployment-5bd4f98fdb-fvvh2 1/1     Running   0           73m
```

Scale up of pods when treshhold limits are met

```
Every 1.0s: kubectl get pods -o wide                                k8s-master: Mon Jan 12 19:18:01 2020
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE             NOMINATED NODE   READINESS GATES
webserver-deployment-5bd4f98fdb-27fg4 1/1     Running   0           83m   192.168.246.193 ip-10-0-0-14    <none>           <none>
webserver-deployment-5bd4f98fdb-7hmzv 1/1     Running   0           18s   192.168.254.140 k8s-worker      <none>           <none>
webserver-deployment-5bd4f98fdb-9tdkc 1/1     Running   0           7m14s 192.168.246.196 ip-10-0-0-14    <none>           <none>
webserver-deployment-5bd4f98fdb-fvvh2 1/1     Running   0           83m   192.168.254.132 k8s-worker      <none>           <none>
webserver-deployment-5bd4f98fdb-n4kjf 1/1     Running   0           17s   192.168.246.200 ip-10-0-0-14    <none>           <none>
webserver-deployment-5bd4f98fdb-xp286 1/1     Running   0           18s   192.168.254.139 k8s-worker      <none>           <none>
webserver-deployment-5bd4f98fdb-zlwrf 1/1     Running   0           18s   192.168.246.199 ip-10-0-0-14    <none>           <none>
```

Self-Managed Cluster

Kubernetes cluster deployed directly on AWS EC2 instances (bypassing EKS).

Custom Autoscaler

Configured for a simple dockerized web application running in Kubernetes pods with limited resource allocation to simulate the scaling of pods to register users and fetch users registered.

Scale-Out/In Validation

Scaling In/Out happens based on the CPU utilization on each pod when CPU surpasses the CPU Threshold of 50% Kubernetes Autoscaler spawns new pods among available Kubernetes worker nodes.

Scaling In/Out of Kubernetes worker nodes happen when Cloud watch monitoring hits 80% CPU across available Kuberentes Worker nodes it spawns new Kubernetes worker node to increase compute resources.



Browser-Based User Interface

Deployment of a complex multi-tier web application on EC2, featuring comprehensive autoscaling for all layers.

Node port round robin

Pods gets even load from the
Kubernetes Service

Application Tier

Backend logic and processing.

Data Tier

Database instances (example: RDS).



Testing Application Resilience

Rigorous testing to confirm the application's ability to gracefully scale both up and down under varying traffic conditions.

- Load balancing across Kubernetes pods.
- Seamless data cache loading across app instances.
- Performance monitoring and alerts.



Conclusion: Demonstrating Cloud Agility

This project successfully demonstrates various autoscaling strategies and their practical implementation within AWS environments, highlighting the agility and efficiency of cloud computing.

- Adaptive Algorithm Verified
- Lambda Scalability Proven
- Kubernetes Autoscaling Confirmed
- Multi-Tier Resilience Validated